

データベース演習

第3回：論理設計とパフォーマンス，DB作成

鈴木源吾

前回の復習

1. 解説：システム開発とデータベース設計
 - システム開発の工程
 - 設計工程とデータベース
2. 例題：Google Colabを使ったデータベース作成・検索
3. 解説：データベースの設計
4. 演習：大学データベースの作成

今回のトピック

1. 解説：パフォーマンスとテーブルの非正規化
2. 解説：インデックスの設計と作成
3. 演習：インデックスの作成と効果の確認

1. 解説：パフォーマンスとテーブルの非正規化

論理設計と物理設計

論理設計・物理設計でやるべきことの基本を演習し，データベースの作成までを行う

1. テーブル・カラムの決定
2. データ型の決定
3. 参照整合性の決定（主キー・外部キーの設定）
4. テーブルの作成
5. インデックスの作成
6. データの作成

今回は，主に5を実施する

システムの非機能要件とデータベース設計

- データベース設計では、パフォーマンス（例：応答性能）や可用性（例：稼働時間）などの非機能要件の考慮も必要となる
- 今回は、その中でも基本的なパフォーマンスに影響する設計を学ぶ
- パフォーマンスを意識する設計段階は、主に物理設計（下図は第2回資料より）

	データモデル	特徴	やること（例）	アウトプット（例）
概念設計	ERモデル	DBMSに依存せず実世界を理想的に記述	ER図の作成	ER図
論理設計	リレーショナルモデル	DBMSも合わせた論理的構造を記述	テーブル・カラム・データ型の決定	テーブル設計書（概念スキーマ）
物理設計	リレーショナルモデル	実際に物理的に格納・管理するための記述	インデックス付与, 分散・冗長化, 格納領域・パラメータ決定	テーブル作成SQL・パラメータ設定, 等（内部スキーマ）

注：3層スキーマのうち外部スキーマはアプリケーションに依存するため、データベース設計のアウトプットに含めなかった

今回扱わないパフォーマンス論点

パフォーマンスに影響する要因は多岐にわたる．今回扱う「非正規化」「インデックス設計」以外にも，実務上は以下のような重要な要因がある

【DB単体に関する論点】

- ストレージの冗長構成（RAID）／ファイルの物理配置
- 接続プール（pgBouncer等），共有バッファ・キャッシュ設定
- パーティショニング（テーブルを物理的に分割して管理）

【システム全体での論点】

- 読み取りレプリカ・シャーディング（複数台へのスケールアウト）
- キャッシュ層（Redis等）でDBへの問合せ自体を減らす

→ いずれも今回は割愛．興味のある人は参考書を参照すること

正規化の振り返り

リレーショナルデータモデルの正規化は，様々な更新時異状を発生させないために必要であると，データベースの基礎で学んだ

- 更新時異状とは，例えば，一つのデータを削除したときに，同時に他の事実も失われてしまうような状況だった
- それを防ぐために，リレーションを分解し，一つのもの（実体）は一つのリレーションだけに入るようにした（正規化）

正規化されたテーブルの例

- 下図は典型的な正規化されたテーブルである
- 会社・社員・部署という実体が、きちんと分解されている
- それぞれのテーブルの主キーに下線をひいた
(社員的主キーは、会社コードと社員IDの複合キー)

会社

<u>会社コード</u>	会社名
C0001	A 商事
C0002	B 化学
C0003	C 建設

社員

<u>会社コード</u>	<u>社員ID</u>	社員名	年齢	部署コード
C0001	000A	加藤	40	D01
C0001	000B	藤本	32	D02
C0001	001F	三島	50	D03
C0002	000A	斉藤	47	D03
C0002	009F	田島	25	D01
C0002	010A	渋谷	33	D04

部署

<u>部署コード</u>	部署名
D01	開発
D02	人事
D03	営業
D04	総務

正規化されたテーブルに対する問合せ

正規化されたテーブルから情報を得ようとすれば、複数のテーブルを結合する操作が増えることになる

例) 田島さんの勤めている会社と部署を知りたい

```
SELECT 会社.会社名, 部署.部署名  
FROM 社員, 会社, 部署  
WHERE 社員.社員名 = '田島' AND  
      社員.会社コード = 会社.会社コード AND  
      社員.部署コード = 部署.部署コード;
```

→ 結合の増加 → 性能劣化の可能性あり

- 結合は非常にコストの高い操作. 多いテーブル数の結合は、レコード数が増える
と性能が劣化する

非正規化による解決

性能の劣化を避けるために、正規化を諦めることが実用上はある。

これを**非正規化**と呼ぶ。下のテーブルの場合、前の問合せは以下ですむことになる

```
SELECT 会社名,部署名 FROM 社員 WHERE 社員名 = '田島';
```

社員

会社コード	会社名	社員ID	社員名	年齢	部署コード	部署名
C0001	A 商事	000A	加藤	40	D01	開発
C0001	A 商事	000B	藤本	32	D02	人事
C0001	A 商事	001F	三島	50	D03	営業
C0002	B 化学	000A	斉藤	47	D03	営業
C0002	B 化学	009F	田島	25	D01	開発
C0002	B 化学	010A	渋谷	33	D04	総務

非正規化による情報の損失

しかし，p.10のテーブル 社員には，p.8のテーブル 会社に含まれていた「会社コード C0003である会社の会社名がC建設」であるという情報が損失している．

- データベースの基礎で学んだ，更新時異状に相当
- テーブル 社員に，C建設の社員のレコードがないことが原因
- かといって，(C0003, C建設, null, ..., null,) という情報を追加すれば，社員IDが主キー（の一部）である制約に矛盾してしまう

→ **非正規化は，情報を損失なく，整合性を保って維持するには，望ましくない**

注：非正規化した社員と合わせて、会社テーブルも保持しておけばこの問題は発生しない

正規化・非正規化と更新の関係

一方、正規化・非正規化と更新との関係はどうなるか？

→ 正規化：1つのレコード更新でよい，非正規化：多くのレコードの更新要

例：A商事の会社名がE物産に変更

正規化

```
UPDATE 会社 SET 会社名 = 'E物産' WHERE 会社名 = 'A商事';
```

非正規化

```
UPDATE 社員 SET 会社名 = 'E物産' WHERE 会社名 = 'A商事';
```

正規化・非正規化の判断は？

エンジニア・理論家によって意見のわかれるところ

- ある人：正規化は必須で、いかなる理由があっても非正規化すべきでない
- 別の人：最初から非正規化を論理設計の中に織り込むべき

データ整合性とパフォーマンスのトレードオフだが・・・

データベースの大家、クリス・デイトによると（参考書より）

「非正規化」はあくまでも最後の手段

非正規化の誘惑にかられても、他の手段によってパフォーマンス向上が図れないかを最後の最後まで検討すべき

2. 解説：インデックスの設計と作成

インデックスによる性能向上

インデックスはデータベースの基礎で学んだ (B+木). インデックスはSQLのパフォーマンス向上のための非常にポピュラーな手段. 理由を以下にあげる

- アプリケーションのコードに影響を与えない
 - データベース側にインデックスを作成すればよいだけ
- テーブルのデータに影響を与えない
- それでいて性能向上の効果が大きい
 - n (フルスキャン) から $\log n$ (ツリーの高さ) のオーダーに向上

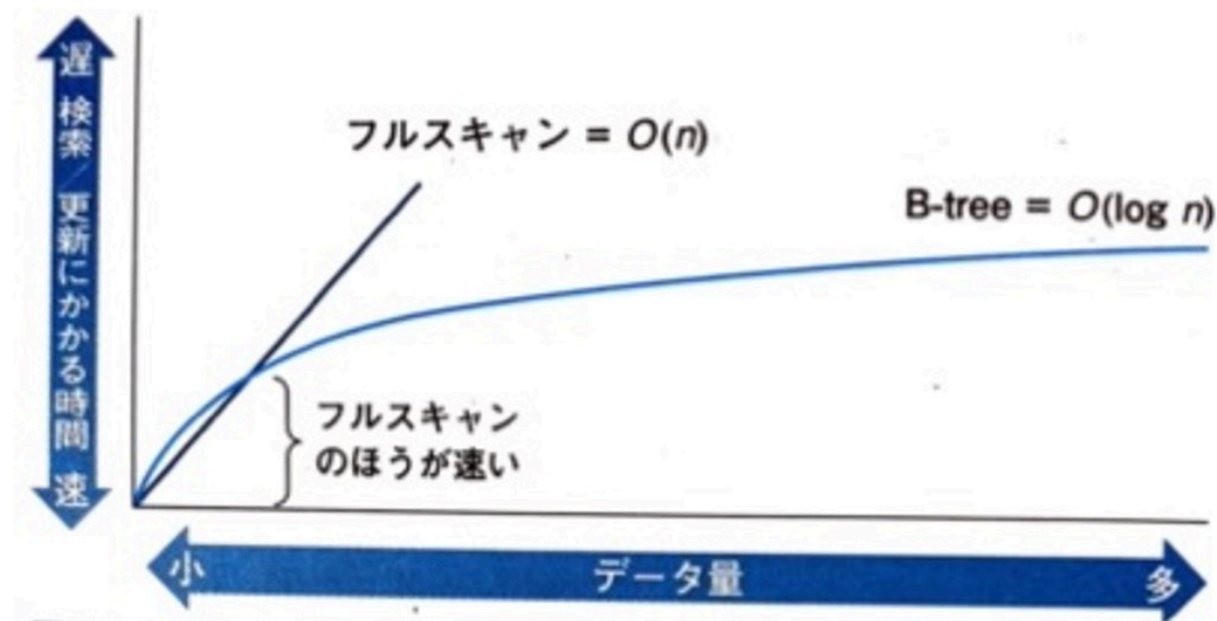
古典的な3つの設計指針

参考書などで古くから紹介されてきた代表的な指針

1. 大規模なテーブルに対して作成する
 2. 値の種類が「適度に」多いカラムに作成する
 3. WHERE句・結合条件に使われるカラムに作成する
- 長く使われてきた指針で，今でも**出発点**として有効
 - ただし，マシン性能やDBMSの進化で前提が変わっている部分もある
 - まず古典的指針を確認し，そのあとで現代的な観点を補足する

古典的指針1：大規模テーブルに作成

- データ量が小さい場合は、フルスキャンの方が速いケースがある（下図）
 - オプティマイザ（最適化器）がインデックスを使うかどうかを判断する
- データが主記憶にすべて乗るくらい小さければ、フルスキャンで十分速い
 - 古典的には「レコード数1万以下なら効果なし」と言われた（参考書）



古典的指針2：「適度な」カーディナリティ

値の種類（カーディナリティ）が「適度」であることが望ましいとされた

- 値の種類が少なすぎる例：性別のような2値（男性・女性）
 - インデックスのツリーは小さく，効果が少ないとされてきた
- 自由文字列の例：備考のような長いテキスト
 - 等価検索の対象になりにくい．全文検索の活用を検討すべき
- 古典的な目安：特定のキー値で全レコードの5%程度に絞り込めれば効果あり
 - 受付日のようなカラムは，365日のうち1日を指定すれば0.3%に絞り込める

古典的指針3：WHERE・結合条件に使われる列

- インデックスが使われるには、問合せの中で条件値が指定されなければならない
- 古典的に「インデックスが使われにくい」とされたパターン
 - LIKE句の中間一致検索（LIKE '%検索文字列%'）
 - ORを用いたWHERE句
 - カラムに演算や関数を適用している場合（例：YEAR(受付日) = 2025）

補足：主キーにはほとんどのDBMSで自動的にインデックスが作成されるため、別途追加する必要なし

（PostgreSQLの場合，pg_indexesテーブルで確認できる）

古典指針の数値目安は時代で変わる

古典的指針が成立したのはHDD・小容量メモリの時代．ハードウェアの進化で前提も変わる

- SSD/NVMeで読み取りが高速化．メモリも大容量化
 - 「フルスキャンが勝つ」テーブルサイズの閾値が上がっている
 - 「5%以下に絞り込めれば」という目安も緩んでいる
- 「1万件」「5%」のような**具体的な数値はあくまで当時の目安**
 - 同じ考え方は通用するが、閾値そのものを鵜呑みにしない

→ 古典的指針は出発点．**最終的にはオプティマイザの判断とEXPLAINでの実測が頼り**

補足の観点1：書き込みコストとのトレードオフ

インデックスは「タダ」ではない．読み取りを速くする代償として

- INSERT/UPDATE/DELETE が遅くなる
 - テーブル更新と同時にインデックスも更新する必要があるため
- ストレージ容量を消費する
- インデックスが多いほど，オプティマイザの選択肢が増えコストも上がる

→ 「とりあえず全カラムにインデックス」は悪手．読み取り頻度・書き込み頻度のバランスで判断する

補足の観点2：複合インデックスのカラム順序

複数カラムにまたがる複合インデックスは、**カラムの順序**が性能に直結する

- 例：`CREATE INDEX ON 社員 (会社コード, 部署コード)`
 - `WHERE 会社コード = ...` → 使える
 - `WHERE 会社コード = ... AND 部署コード = ...` → 使える
 - `WHERE 部署コード = ...` のみ → **使えない**（先頭カラムが指定されないため）
- 設計の考え方
 - よくWHEREに現れるカラムを先頭に置く
 - 選択性の高い（=絞り込める）カラムを優先する

現代的な観点：DBMSの機能で「不向き」を打破

PostgreSQLには、古典的に「インデックス不向き」とされたケースに対応する機能がある

- **LIKE中間一致** → 全文検索系拡張 (pg_trgm + GINインデックス)
- **関数・式の適用** → 式インデックス (例： `CREATE INDEX ON 受付 ((YEAR(受付日)))`)
- **特定の条件のみ高速化したい** → 部分インデックス (WHERE句付きCREATE INDEX)
- **2値などの低カーディナリティ** → 部分インデックスや複合インデックスの一部として活用
- **インデックスのみで結果を返したい** → カバリングインデックス (INCLUDE句)

→ 「不向き」と決めつけず、**DBMSの機能を調べる**ことが現代的な姿勢

設計の進め方：仮説と実測の往復

古典的指針も現代的観点も、あくまで**仮説を立てる材料**

1. クエリと使われ方を見て、候補となるインデックスを設計する
2. 実際にインデックスを作成して、**EXPLAINで実行プランを確認**する
3. 想定通りに使われない／効果がなければ再設計する

→ 設計者は「正解」を一発で出すのではなく、DBMSと対話しながら最適化していく

実務での進め方：事前の目星と事後の拾い上げ

すべてのクエリをEXPLAINで検証するのは現実的でない．実務では二段構えが基本

事前（設計・実装時）：要件から目星をつける

- 高頻度のクエリ（ログイン，検索，一覧表示）
- 大量データを扱うクエリ（夜間バッチ，集計レポート）
- SLAが厳しいクエリ（決済，ユーザ向けAPI）

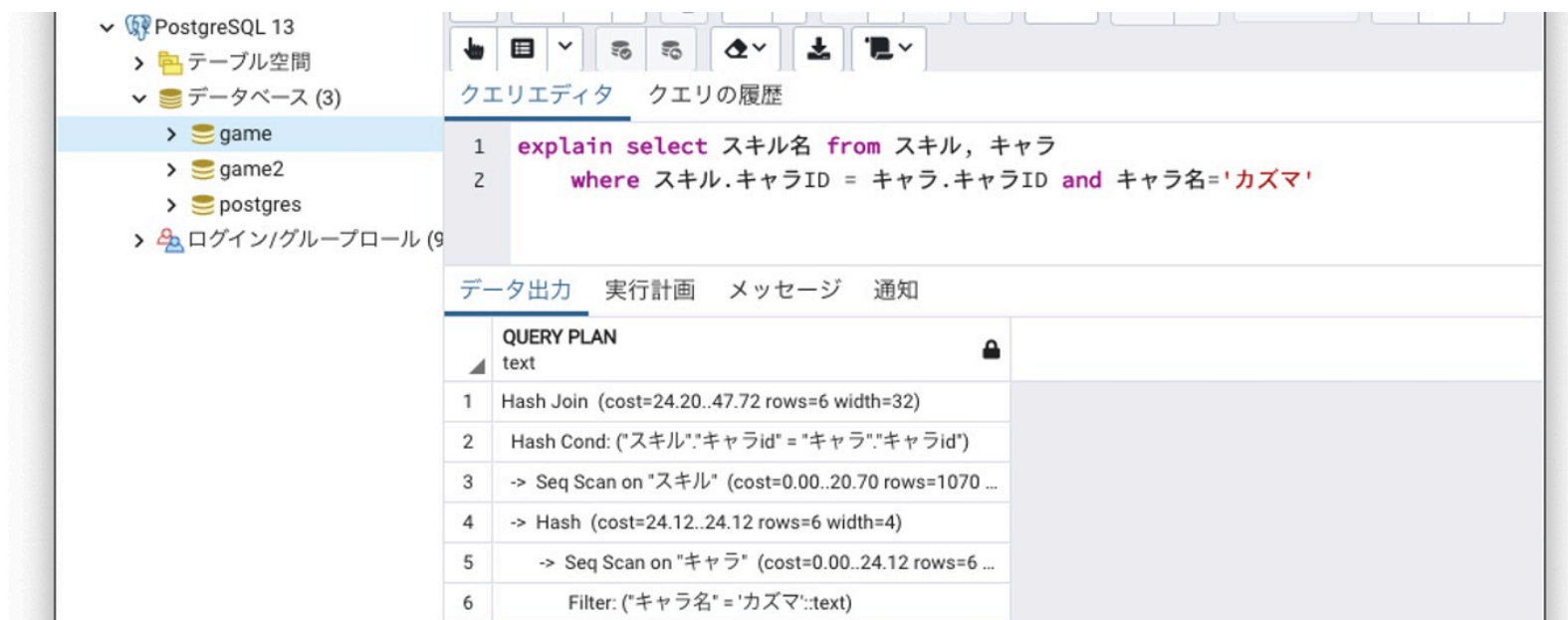
事後（運用時）：実測から拾う仕組みを入れる

- `pg_stat_statements` 拡張でクエリごとの実行時間・回数を集計
- `log_min_duration_statement` でしきい値を超えたSQLをサーバログに記録
- 監視・APM（アプリ性能監視ツール）でスローエンドポイントから逆引き

→ 設計時の全網羅は諦め、**遅いものを拾う仕組み**を運用に組み込んでおく

インデックス使用を確認するためには？

PostgreSQLでは、EXPLAINというコマンドが用意されており、EXPLAIN SQL文、と指定することで、実行プラン（SQL文が変換されたデータアクセスの手続き）を出すことができる（以下、実行例．Seq Scan→Sequential Scan（順スキャン）なのでインデックスは使われていない）



The screenshot shows a PostgreSQL client interface with a tree view on the left and a query editor on the right. The tree view shows the database structure, including a database named 'postgres' and a table named 'game'. The query editor shows the following SQL query:

```
1 explain select スキル名 from スキル, キャラ
2 where スキル.キャラID = キャラ.キャラID and キャラ名='カズマ'
```

Below the query editor, the 'データ出力' (Data Output) tab is selected, showing the 'QUERY PLAN' (Query Plan) for the executed query. The plan is as follows:

Step	Operation	Cost	Rows	Width
1	Hash Join	(cost=24.20..47.72)	rows=6	width=32
2	Hash Cond: ("スキル".キャラid = "キャラ".キャラid)			
3	-> Seq Scan on "スキル"	(cost=0.00..20.70)	rows=1070	
4	-> Hash	(cost=24.12..24.12)	rows=6	width=4
5	-> Seq Scan on "キャラ"	(cost=0.00..24.12)	rows=6	
6	Filter: ("キャラ名" = 'カズマ':text)			

インデックスの作成（CREATE INDEX）

CREATE INDEX文を使ってインデックスを生成することができる

基本となる書式

```
CREATE INDEX [ インデックスの名前 ] ON [ ONLY ] テーブル名  
    ( カラム名 [, ...] )
```

- カラムを複数指定したインデックスの作成も可能（カラム複数の値に対してインデックスができる．複数の値を条件に指定した検索が高速になる）
- インデックスが作成されたことは，pg_indexesテーブルを参照すればわかる

実行例：テーブル スキルのカラム スキル名にskill_name_idxという名前のインデックスを作成する

```
CREATE INDEX skill_name_idx ON スキル (スキル名)
```

3. 演習：インデックスの作成と効果の確認

第3回演習のノートブックに従って実施すること

課題1

第2回・第3回の演習のノートブックを実施し，WebClassに共有リンクで提出すること